CincyDeliver.org

Cincy Deliver 2026 Sponsors

Diamond

ingage

The Japikse Family Foundation

Platinum

((CENTRIC))

Gold

Vindex

CINCY DELIVER 2026 · JULY 24, 2026

Agents Are Products, Not Prompts

Operationalizing AI-assisted development beyond hero developers

> *The discipline we already apply to code — version control, review, testing, lifecycle, deprecation — is exactly what agents need.*

SHAWN WALLACE

Principal Architect · Centric Consulting

THE COLD OPEN

One engineer's practice worked — for one person

Built alone

No design review, no team input — refined solo until it worked exactly right. Genuinely good work.

Inherited, not understood

The team got the assets, not the judgment. Nobody had shaped the constraints or earned the trust.

Adoption stalled

Nobody trusted output they hadn't helped build. The work was good — the practice wasn't shared.

The model didn't fail — the practice was never designed to be handed off.

Promise: leave able to build agents you can maintain, test, hand off, and change without fear.

AI is good at code. Not software.

Code vs. system

AI accelerates boilerplate and execution. Structure, boundaries, and “is it done / is it correct” are still human judgment.

The split no one names

AI amplifies seniors with taste — and quietly leaves juniors shipping code that looks senior without understanding it.

The hero trap

If only a few seniors know when to trust the output, that judgment is trapped in their heads. It doesn't scale or develop the next generation.

Real wishful thinking

Hoping this works itself out. The alternative: operationalize — encode the judgment into artifacts the whole team uses.

FRAMEWORK

Agentic AI Maturity Model

Coding & Application Modernization

THE CAPABILITY GAP

LEVEL 1

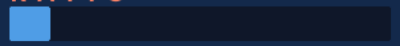
AI Autocomplete

INTEGRATION

IDE Plugin

Inline suggestions; integration beyond the editor is rare.

AI / HUMAN RATIO



10% AI 90% Human

PRODUCTIVITY

~10-15%

Producer

LEVEL 2

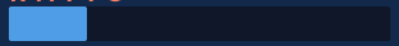
AI Chat

INTEGRATION

Chatbot / Sidebar

Conversational help: explain, refactor, generate snippets.

AI / HUMAN RATIO



20% AI 80% Human

PRODUCTIVITY

~20-25%

Producer

LEVEL 3

AI Assist

INTEGRATION

Copilot / IDE Agent

Multi-step edits across files; human reviews and ships.

AI / HUMAN RATIO

50% AI 50% Human

PRODUCTIVITY

~25-50%

Reviewer

LEVEL 4

Agentic Augmented

INTEGRATION

Long-Running Agents

Background tasks, PR creation, narrated cloud workflows.

AI / HUMAN RATIO

75% AI 25% Human

PRODUCTIVITY

~1x-5x

Director

LEVEL 5

Agentic Adaptive

INTEGRATION

Agents as Automation

Self-directing systems; humans set goals, review outcomes.

AI / HUMAN RATIO

90% AI 10% Human

PRODUCTIVITY

~10-20x

Director

Individual Productivity

The human role shifts from Producer → Reviewer → Director

Organizational Capability

PROMPT ENGINEERING ≠ SOFTWARE ENGINEERING

One prompt away from “working” — so teams stop there

Inconsistent

Same input, different behavior.

Unmaintainable

One giant prompt — no named parts.

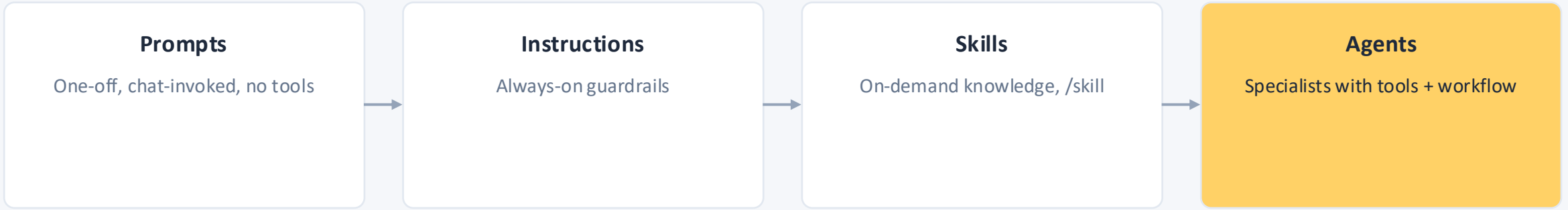
Brittle

Requirements shift, it quietly breaks.

Common pitfalls: task-based agents · vague instructions · over-scoping · no testing · set-and-forget

THE FRAMEWORK FOR ENGINEERING AN AGENT

Prompts → Instructions → Skills → Agents



The same ladder as the maturity model — agents are the engineered end.

DESIGN RIGOR APPLIES HERE TOO

Right layer compounds. Wrong layer costs you.

Instructions

Right — small, non-negotiable, loaded on every task.
Wrong — bloat it and every call gets slower and vaguer.

Skills

Right — real procedure, loaded only when triggered.
Wrong — bury it and it never fires when it's needed.

Agents

Right — enough scope to justify its own context and tools.
Wrong — spin one up too small and it's dead weight to maintain.

Same rigor as any interface: get the layer right and it compounds — get it wrong and it costs you.

A SYSTEM WITH NAMED PARTS, NOT A PARAGRAPH

The Agent Components model

Identity & Role	Who the agent is
Responsibilities	What it will / won't do
Context	Stack rules, known constraints
Constraints	ALWAYS / NEVER rules
Process	Deterministic, repeatable steps
Output Format	Fixed, structured sections
Tone	How it communicates

Plus frontmatter — name, description, tools, model, handoffs.

DESIGN PROCESS, NOT PROMPT TINKERING

Define → scope → design the output

Define
Problem + success criteria

→

Identify role
WHO the agent is

→

Set scope
+ explicit out-of-scope

→

Design output
Fixed, structured format

Role-based scope
WHO not WHAT.

Explicit constraints
ALWAYS / NEVER rules.

Structured outputs
Defined sections.

You don't have to invent this from a blank page — the industry has already converged on this pattern.

LIVE DECOMPOSITION

architecture-reviewer.agent.md, component by component

Frontmatter	name, description, tools: ['read','search/changes'], model
Identity & Role	“Expert software architect specializing in Clean Architecture and DDD.”
Responsibilities	Bounded list of what it will / won't do
Context	.NET layer rules, dependency direction, known violations
Constraints	ALWAYS check circular deps · NEVER break boundaries
Analysis Process	Deterministic, repeatable review steps
Output Format	Summary / Findings / Recommendations, PR-ready
Tone + Examples	Direct, constructive — concrete violation catalog

```
---
name: "agent-name"
description: 'Brief description'
tools: [changes]
model: Claude Sonnet 4.5
---

# Agent Name

[Role and expertise]

## Responsibilities
## Context
## Constraints
## Analysis Process
## Output Format
## Tone
```

The payoff: adding a violation bullet to Context is a one-line change — nothing else breaks.

DEMO

architect-reviewer.agent

PROOF IT'S REAL

What the agent produced

The violation

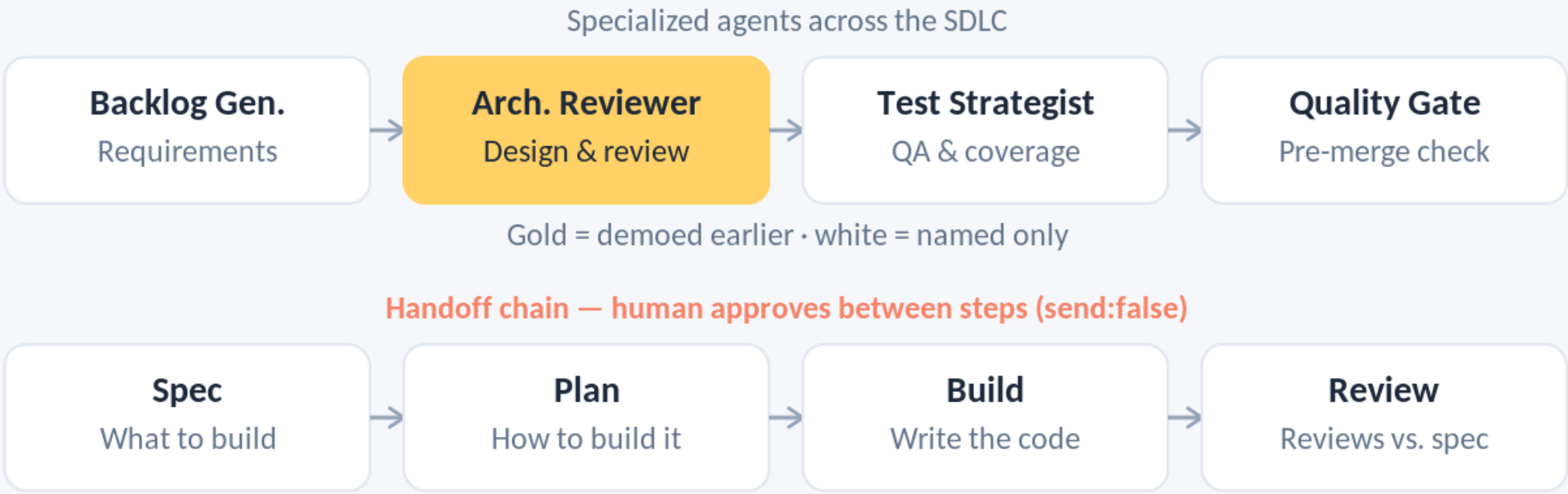
Instantiates SmtpEmailSender directly and imports Infrastructure into Application — small enough to read in seconds, the violation obvious once flagged.

```
architecture-reviewer — NotificationService.cs

Summary: 1 HIGH violation

[HIGH] Application → Infrastructure dependency
  NotificationService instantiates SmtpEmailSender directly (line 12)
  Rule: Application MUST NOT reference Infrastructure

Recommendation: inject IEmailSender; bind the SMTP implementation in the composition root
```

Agents don't make up for a bad process — they amplify a good one. Rigor keeps the artifact aligned with the process, with clear demarcation between phases.

REAL AGENTS ACROSS THE SDLC

Same architecture, different configuration



Architecture Reviewer is the only one demoed — already shown a few minutes ago. The rest: named, not opened.

RUN YOUR AGENTS LIKE A CODEBASE

Apply Your Software Engineering Process

Version control

.agent.md in git; conventional commits; BREAKING CHANGE markers when the output contract changes.

Code review

Every change via PR. Tiered: minor / moderate / major, scaled to blast radius.

Testing

3–5 saved test scenarios per agent; re-run on every change to catch regressions.

Lifecycle

Proposal → development → review → production → maintenance → deprecation. Team-owned.

Deprecation

Retire agents like dead code: mark deprecated, point to replacement, grace period, remove cleanly.

Commit 6f995a3 — walked live via git show

```
feat(agent): add dual-stack support
fix(architecture-reviewer): tighten
  DDD boundary check

BREAKING CHANGE: output schema
  now requires 'Layer' field
```

Minor	Typo — no review
Moderate	New constraint — 1 approval
Major	Role/output change — 2 approvals + docs + announce

This is why the cold-open practice wouldn't have stayed trapped in one person's head — governance is what lets it survive the hand-off, without heroes.

DEMO

git show 6f995a3 — the real commit that added dual-stack support

WHAT TO DO MONDAY

Define → Test → Observe → Refine → Repeat

- Don't start from scratch — fork an open-source agent/skill library (agent-skills, Squad, hve-core)
- Define the contract before the prompt
- Decompose into named components
- Write 3–5 test scenarios
- Commit as a reviewed .agent.md
- Iterate on components, not one monolithic prompt

FURTHER READING

Open-source pointers, and where to find me

agent-skills

addyosmani — production-grade skills.
github.com/addyosmani/agent-skills

Squad

bradygaster (MS) — agent teams as files.
github.com/bradygaster/squad

hve-core

Microsoft — agents, skills + RPI.
github.com/microsoft/hve-core

Awesome Copilot

Curated Copilot customizations.
awesome-copilot.github.com

Twitter / X

Shawn Wallace
x.com/ShawnWallace

LinkedIn

Shawn Wallace
linkedin.com/in/shawnwallace

GitHub

Shawn Wallace
github.com/shawnewallace

Don't start from scratch — fork one of these, then treat it like the rest of this talk.

THANK YOU

> **The senior engineer's practice didn't need a smarter model.**
>
> ***It needed to stop living in one person's head.***

Design your agents. Maintain them like code. Operationalize the judgment — that's how the human role becomes Director instead of hero producer.

SHAWN WALLACE

Principal Architect

Centric Consulting

github.com/shawnewallace

Slides, checklist, and agent files: see session page

CincyDeliver.org

Cincy Deliver 2026 Sponsors

Diamond

ingage

The Japikse Family Foundation

Platinum

((CENTRIC))

Gold

Vindex